

Appearance Manifold 개발자 가이드

런타임 풍화 보간 시스템의 코드 구조, 데이터 포맷, 한계 및 개선 방향

이 문서의 대상

플러그인을 확장하거나 유지보수하는 contributor를 위한 기술 문서입니다. Weathering Manager Component의 사용자 흐름을 C++/Python 구현과 연결해 설명합니다.

1. 시스템 책임 범위

GammaTon 모듈은 풍화 텍스처를 생성하는 에디터 시뮬레이터이고, AppearanceManifold 모듈은 생성된 풍화 상태를 7D appearance space로 다루며 trajectory 생성과 런타임 보간을 담당합니다.

런타임 경로의 핵심은 key texture를 7D tensor로 변환하고, 각 픽셀을 trajectory sample에 투영한 뒤 alpha 값에 따라 중간 tensor를 생성해 transient UTexture2D로 복원하는 것입니다.

2. 모듈 및 파일 구조

파일	역할
AppearanceManifold.cpp	모듈 Startup에서 Python script path를 등록하고 numpy/scipy/scikit-learn 의존성을 확인합니다.
Blueprint Function Library	텍스처 변환, 바이너리 로드, 캐시 생성, 보간, 텍스처 적용을 담당하는 Blueprint 노드 집합입니다.
Build Nearest Trajectory Async	비싼 nearest trajectory 검색을 ThreadPool에서 수행하고 완료 이벤트를 GameThread로 반환합니다.
MainManager.py	Editor 전용 trajectory 생성 파이프라인의 오케스트레이터입니다.
Utility / KNN / MDS / Trajectory	7D tensor 구성, KNN graph, geodesic distance/MDS, trajectory 계산을 담당합니다.
_Reconstructor.py	런타임 보간 알고리즘의 Python 원형입니다.

3. 데이터 모델

모든 런타임 풍화 상태는 픽셀당 7개의 float 값으로 표현됩니다.

채널	의미
0, 1, 2	BaseColor R, G, B
3, 4, 5	Specular R, G, B
6	Roughness
배열 offset	픽셀 i 의 시작 위치는 $i * 7$ 입니다.

4. 바이너리 포맷

현재 구현 기준

런타임 trajectory 바이너리는 $[T:\text{int32}][\text{float} * T * 7]$ 구조입니다. 기존 문서의 $[T][N]$ 헤더 설명은 export_to_unreal_binary 계열의 확장 포맷에 가깝고, 현재 LoadWeatheringBinaryData가 읽는 포맷과 구분해야 합니다.

필드	설명
T	trajectory sample 개수
payload	T개의 7D sample이 float32로 연속 저장됩니다.
로드 함수	LoadWeatheringBinaryData(FilePath, OutT)
생성 함수	MainManager.py의 export_to_unreal_runtime_data

5. 런타임 호출 체인

- 1 LoadWeatheringBinaryData로 trajectory sample과 T를 로드합니다.
- 2 CombineTextureSources 또는 LoadWeatheringTensorsFromFiles로 key texture를 7D tensor로 변환합니다.
- 3 BuildNearestTrajectoryCacheAsync로 각 픽셀의 nearest trajectory index를 캐시합니다.
- 4 InterpolateWeatheringCached로 alpha 기반 중간 tensor를 생성합니다.
- 5 ReconstructTexturesFromTensor로 transient UTexture2D 3종을 생성 또는 재사용합니다.
- 6 ApplyWeatheringTexturesToMesh로 MID의 Base/Spec/Rough 파라미터에 텍스처를 적용합니다.

6. 보간 알고리즘

각 픽셀은 key texture A와 B에서 각각 trajectory의 가장 가까운 sample index를 갖습니다. Alpha가 주어지면 두 index 사이의 중간 index를 계산하고, 해당 trajectory sample을 기준으로 A와 B의 색상/물성 delta를 양방향으로 반영합니다.

현재 C++ 구현은 per-channel unrolling과 clamp를 사용합니다. 이는 단순하고 빠르지만, 가우시안 주변 샘플 가중 보간이나 연속적인 sample interpolation까지는 수행하지 않습니다.

7. 성능 특성

단계	비용/특성
BuildNearestTrajectoryCache	$O(\text{PixelCount} * T * 7)$. 가장 비싼 단계이며 비동기 처리 대상입니다.
InterpolateWeatheringCached	$O(\text{PixelCount} * 7)$. 캐시 이후에는 비교적 가볍습니다.
ReconstructTexturesFromTensor	CPU에서 BGRA 배열을 만들고 UpdateTextureRegions로 GPU에 업로드합니다.
ApplyWeatheringTexturesToMesh	MID 생성 또는 재사용 후 텍스처 파라미터를 갱신합니다.

8. 현재 한계

- Nearest search는 brute-force입니다. trajectory 길이와 텍스처 해상도가 커지면 캐시 생성 비용이 급격히 증가합니다.
- InterpolateWeatheringCached는 중간 index를 round하므로 trajectory sample 사이의 완전한 연속 보간은 아닙니다.
- 런타임 텍스처는 transient이며, 에셋 저장이나 네트워크 동기화는 별도 설계가 필요합니다.
- GPU 업로드 비용이 있어 매 프레임 고해상도 텍스처를 갱신하면 프레임 드랍이 발생할 수 있습니다.
- Editor trajectory 생성은 PythonScriptPlugin과 외부 Python wheel 의존성에 묶여 있습니다.
- 문서와 코드 사이에 경로 및 바이너리 포맷 설명 차이가 있으므로 contributor는 현재 C++ 로더를 기준으로 확인해야 합니다.

9. 개선 방향

영역	개선 제안
검색 최적화	KD-tree, quantized LUT, trajectory segment acceleration structure로 nearest cache 비용을 줄입니다.
보간 품질	round 대신 trajectory sample 간 선형 보간 또는 주변 sample gaussian blending을 도입합니다.
GPU 경로	Compute Shader 또는 Render Target 기반 업데이트로 CPU 배열 생성과 UpdateTextureRegions 비용을 줄입니다.
캐시 관리	ActorName/Index 기반 파일명 충돌 방지, 버전 헤더, 해상도/채널 메타데이터를 추가합니다.
에러 처리	Blueprint 노드별 실패 원인을 반환하거나 로그 카테고리를 분리합니다.
패키징 안정성	Editor-only Python 경로와 Runtime-only 로더 경로를 모듈/매크로 수준에서 더 명확히 분리합니다.

10. Contributor 체크리스트

- 7D channel order를 변경했다면 Python export, C++ load, reconstruct 함수를 모두 함께 수정합니다.
- 바이너리 포맷을 변경했다면 version/header를 추가하고 기존 파일 호환성을 검토합니다.
- Texture update 코드를 수정했다면 transient texture 재사용, 크기 변경, GC, GameThread 요구사항을 확인합니다.
- 비동기 노드를 수정했다면 Completed delegate가 GameThread에서 호출되는지 확인합니다.
- Blueprint Component의 노출 변수명을 변경했다면 사용자 문서와 기존 uasset 참조를 함께 갱신합니다.
- 성능 변경은 256, 512, 1024 해상도에서 캐시 생성 시간과 런타임 업데이트 시간을 비교합니다.